

| <b>Particular</b>                    | <b>Details</b>                  |
|--------------------------------------|---------------------------------|
| <b>Department:</b>                   | B.Tech                          |
| <b>Programme:</b>                    | IT                              |
| <b>Course Code &amp; Course Name</b> | CSA0904-Programming in Java     |
| <b>Academic Year / Batch</b>         | 2024                            |
| <b>Faculty Name</b>                  | Dr.R.Thalapthi Rajasekaran      |
| <b>Assignment Title</b>              | Smart Library Management System |
| <b>Date of Issue</b>                 | 01-09-2026                      |
| <b>Date of Submission</b>            | 02-09-2026                      |
| <b>Maximum Marks</b>                 |                                 |
| <b>Course Outcome(s) – CO</b>        |                                 |
| <b>Bloom's Taxonomy Level</b>        |                                 |
| <b>SDG Mapping (SDG 1-17)</b>        |                                 |
| <b>Industry / Societal Relevance</b> |                                 |

## **1. Problem Statement and Problem Formulation**

A library requires an efficient system to manage books, members, borrowing, reservations, overdue notifications, and report generation. Manual record maintenance is time-consuming and prone to errors such as duplicate entries, inaccurate inventory, and delayed notifications.

The proposed Smart Library Management System is a Java Applet-based GUI application that automates these activities using Object-Oriented Programming, Java Collections Framework, Exception Handling, and Multithreading. The system maintains book availability, supports reservations through a waiting list, generates reports, and sends due-date and overdue notifications while ensuring safe concurrent access to shared library resources.

## **2. Objective and Expected Outcomes**

### **Objectives**

- Develop a Java Applet-based GUI application.
- Apply Object-Oriented Programming concepts.
- Manage books and members efficiently.
- Automate book issue and return.
- Implement reservation and notification features.
- Generate circulation and inventory reports.
- Demonstrate multithreading and synchronization.

### **Expected Outcomes**

- Accurate book inventory.
- Fast member registration.
- Automated fine calculation.
- Efficient reservation management.
- Reliable overdue notification.
- Reduced manual errors.

### **3. Requirements, Constraints and Assumptions**

#### **Functional Requirements**

- Add books
- Register members
- Search books
- Issue books
- Return books
- Reserve books
- Cancel reservations
- Display books
- Display members
- Generate reports
- Send notifications

#### **Constraints**

- Java Applet GUI Application
- JDK 17
- CSV files used as persistent database
- Single administrator

#### **Assumptions**

- Book ID is unique.
- Member ID is unique.
- Fine is calculated only for overdue books.
- Notifications are displayed through the graphical user interface.

#### **4. Application of Relevant Course Knowledge**

The project demonstrates the following Java concepts:

- Classes and Objects
- Encapsulation
- Data Hiding
- Constructors
- Inheritance
- Polymorphism
- Method Overriding
- ArrayList
- HashMap
- HashSet
- Iterator
- Exception Handling
- User-defined Exceptions
- Multithreading
- Synchronization
- Generics

#### **5. Design / Proposed Solution / Methodology**

The system is divided into the following modules:

##### **Book Management**

- Add Book
- Search Book
- Display Books

## **Member Management**

- Register Member
- Display Members

## **Transaction Module**

- Issue Book
- Return Book

## **Reservation Module**

- Reserve Book
- Cancel Reservation

## **Notification Module**

- Due Notifications
- Overdue Notifications

## **Report Module**

- Inventory Report
- Borrow Report
- Fine Report

## **6. Algorithm / Pseudocode**

START

Display Main Menu

Repeat

Read User Choice

IF Choice = 1

Add Book

ELSE IF Choice = 2

Register Member

ELSE IF Choice = 3

Search Book

ELSE IF Choice = 4

Issue Book

ELSE IF Choice = 5

Return Book

ELSE IF Choice = 6

Reserve Book

ELSE IF Choice = 7

Cancel Reservation

ELSE IF Choice = 8

Display Books

ELSE IF Choice = 9

Display Members

ELSE IF Choice = 10

Generate Reports

ELSE IF Choice = 11

Send Notifications

ELSE IF Choice = 12

Exit

UNTIL Exit

STOP

## 7. Implementation / Environment / Tools Used

### Software Used

| Tool           | Purpose                                             |
|----------------|-----------------------------------------------------|
| Java JDK 17    | Programming Language and Runtime                    |
| IntelliJ IDEA  | Code Development and Debugging                      |
| Antigravity AI | AI-assisted code generation and development support |
| Windows 11     | Operating System                                    |

### Development Environment

- Programming Language: Java
- JDK Version: 17
- IDE: IntelliJ IDEA
- Operating System: Windows 11
- Application Type: GUI-based Java Applet Application

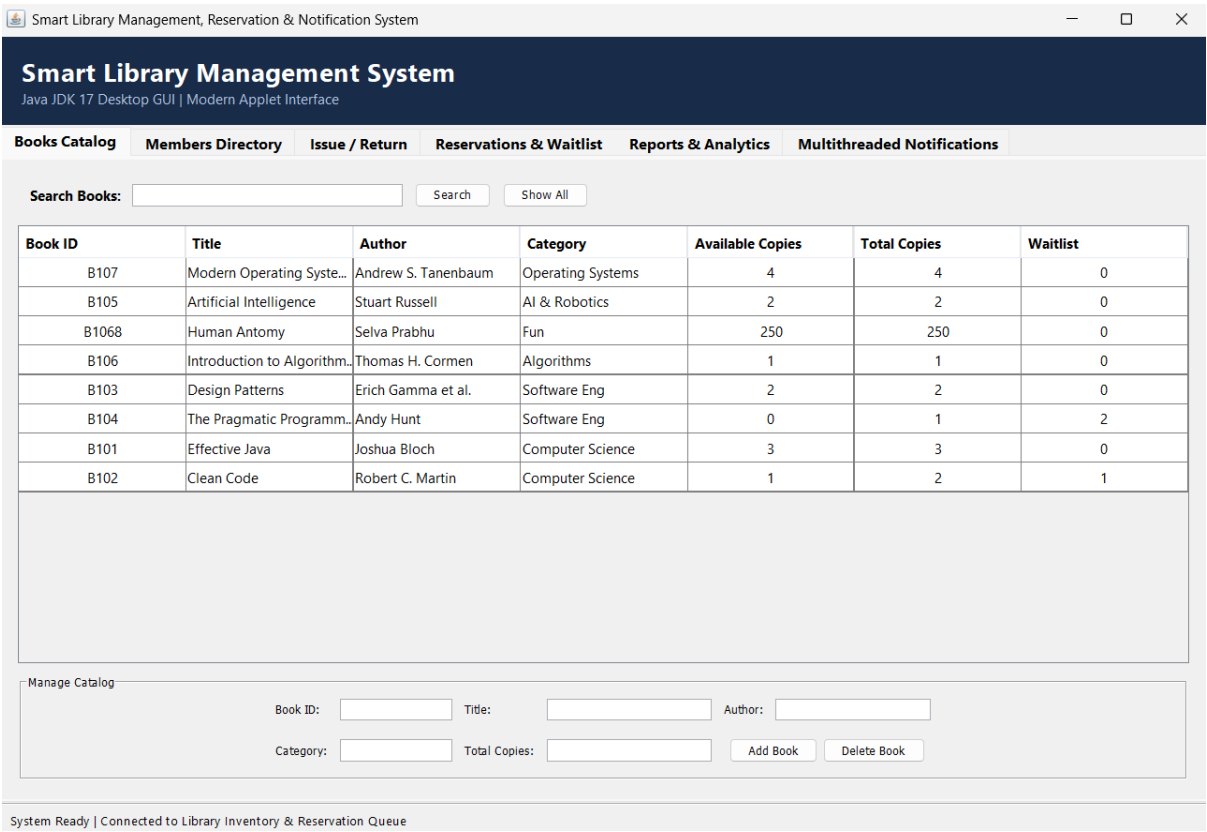
## 8. Test Cases

| Test Case       | Expected Result     | Actual Result       | Status |
|-----------------|---------------------|---------------------|--------|
| Add Book        | Book Added          | Book Added          | Pass   |
| Register Member | Member Registered   | Member Registered   | Pass   |
| Search Book     | Book Found          | Book Found          | Pass   |
| Issue Book      | Book Issued         | Book Issued         | Pass   |
| Return Book     | Book Returned       | Book Returned       | Pass   |
| Reserve Book    | Reservation Created | Reservation Created | Pass   |
| Generate Report | Report Displayed    | Report Displayed    | Pass   |
| Notification    | Message Displayed   | Message Displayed   | Pass   |

## 9. Execution Screenshots / Outputs

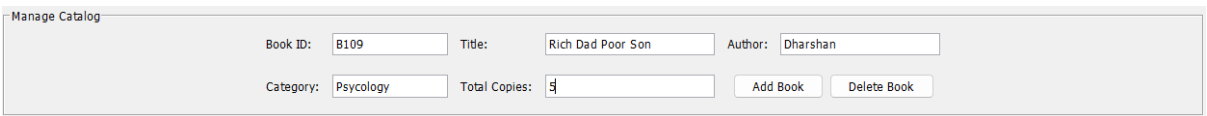
**Figure 1:** Main Menu

The Smart Library Management System starts by loading sample books, members, active loans, and reservation data. A menu is displayed with twelve options, allowing users to perform operations such as adding books, registering members, issuing and returning books, reserving books, generating reports, sending notifications, and exiting the application.



**Figure 2:** Add Book

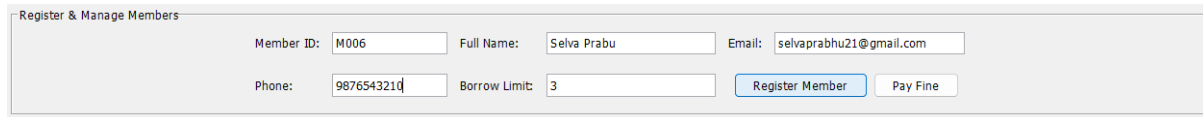
The Add Book module allows the administrator to enter the details of a new book, including Book ID, title, author, category, and the number of available copies. After successful validation, the book is added to the library inventory, and a confirmation message is displayed.





### Figure 3: Register Member

The Register Member module enables the administrator to add a new library member by entering the member ID, name, contact details, and membership type. Once the information is validated, the member is successfully registered in the system.



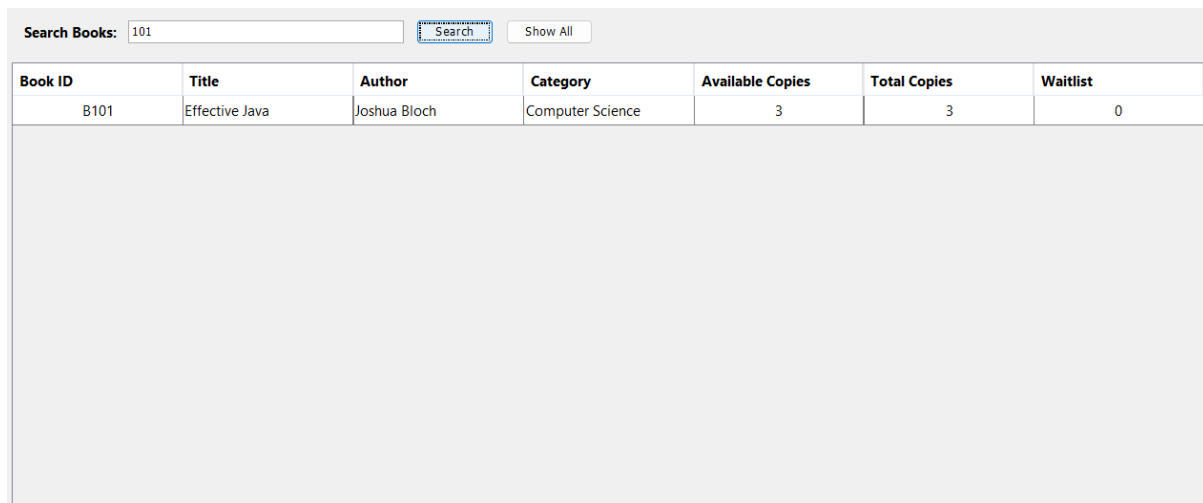
Register & Manage Members

Member ID:  Full Name:  Email:

Phone:  Borrow Limit:

### Figure 4: Search Book

The Search Book module allows users to search for a book using its Book ID or title. If the requested book exists, the application displays complete information including the title, author, category, and availability status.



Search Books:

| Book ID | Title          | Author       | Category         | Available Copies | Total Copies | Waitlist |
|---------|----------------|--------------|------------------|------------------|--------------|----------|
| B101    | Effective Java | Joshua Bloch | Computer Science | 3                | 3            | 0        |

Figure 5: Issue Book

The Issue Book module allows the administrator to issue an available book to a registered member. The system verifies the book and member details, updates the inventory, records the transaction, and displays a confirmation message after successful issuance.

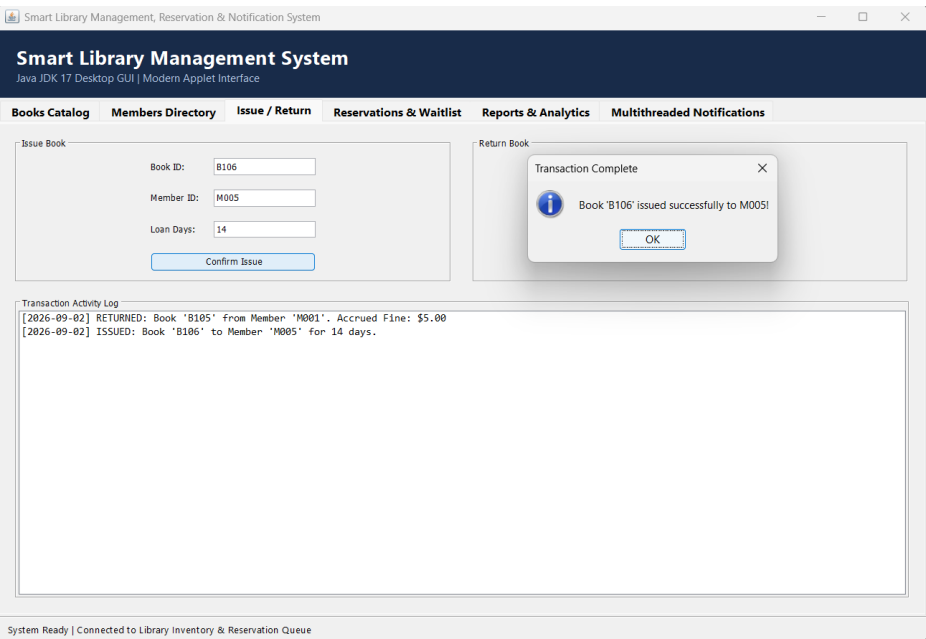


Figure 6: Return Book

The Return Book module processes the return of borrowed books. The application updates the inventory, checks whether the book was returned after the due date, calculates any applicable fine, and confirms the successful completion of the return process.

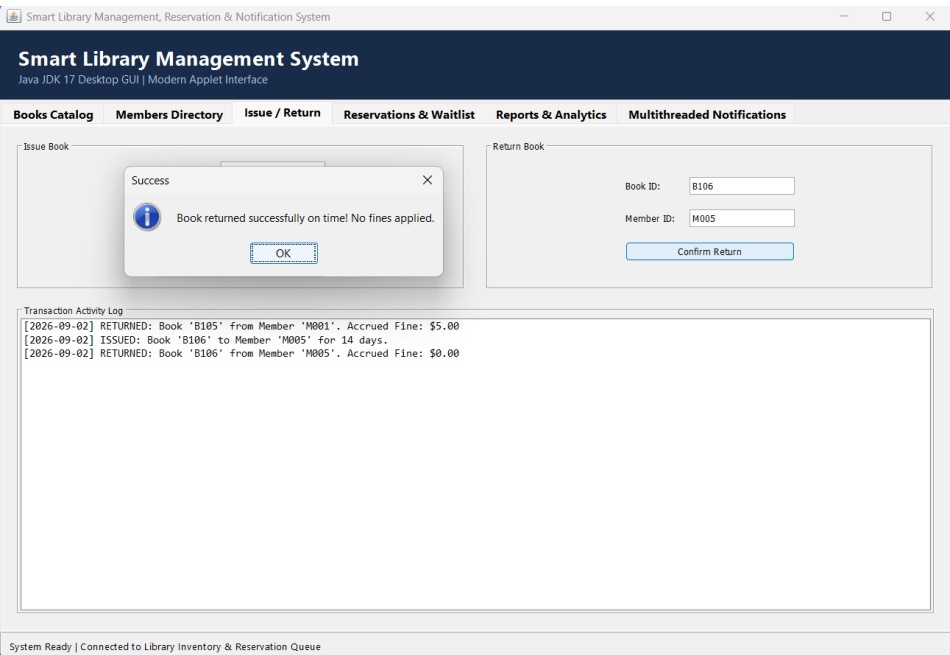


Figure 7: Reserve Book

The Reserve Book module allows members to reserve books that are currently unavailable. The reservation request is added to a waiting list, and the system confirms the reservation for future allocation when the book becomes available.

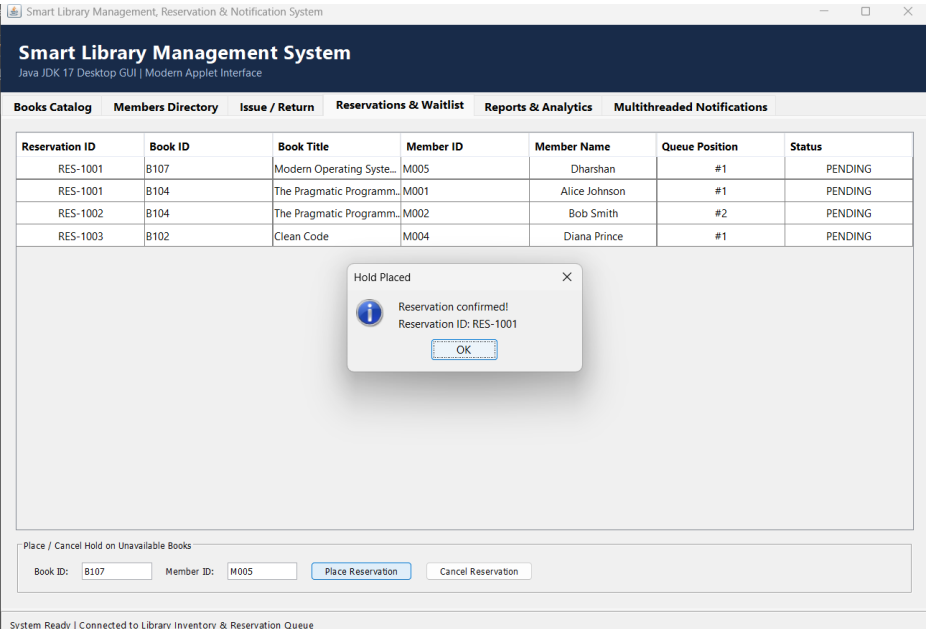
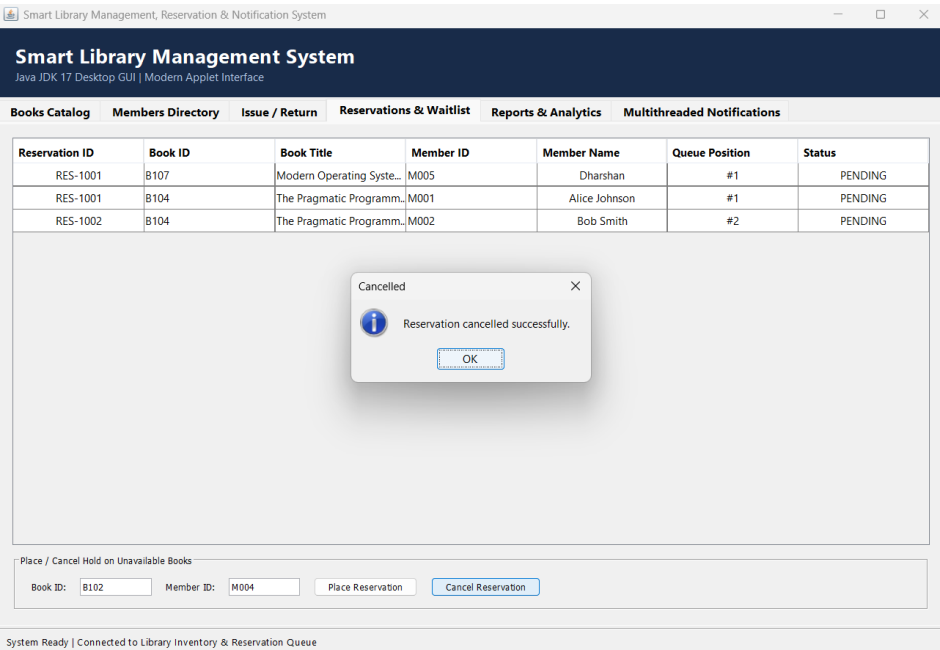


Figure 8: Cancel Reservation

The Cancel Reservation module enables members or the administrator to cancel an existing reservation. The system removes the reservation from the waiting list and updates the reservation records accordingly.



**Figure 9: Display Books**

The Display Books module shows the complete list of books available in the library. The displayed information includes Book ID, title, author, category, available copies, and current availability status, enabling efficient inventory management.

| Book ID | Title                       | Author              | Category          | Available Copies | Total Copies | Waitlist |
|---------|-----------------------------|---------------------|-------------------|------------------|--------------|----------|
| B109    | Rich Dad Poor Son           | Dharshan            | Psychology        | 5                | 5            | 0        |
| B107    | Modern Operating Syste...   | Andrew S. Tanenbaum | Operating Systems | 4                | 4            | 1        |
| B105    | Artificial Intelligence     | Stuart Russell      | AI & Robotics     | 2                | 2            | 0        |
| B1068   | Human Antomy                | Selva Prabhu        | Fun               | 250              | 250          | 0        |
| B106    | Introduction to Algorith... | Thomas H. Cormen    | Algorithms        | 1                | 1            | 0        |
| B103    | Design Patterns             | Erich Gamma et al.  | Software Eng      | 2                | 2            | 0        |
| B104    | The Pragmatic Programm...   | Andy Hunt           | Software Eng      | 0                | 1            | 2        |
| B101    | Effective Java              | Joshua Bloch        | Computer Science  | 3                | 3            | 0        |
| B102    | Clean Code                  | Robert C. Martin    | Computer Science  | 1                | 2            | 0        |

**Figure 10: Display Members**

The Display Members module lists all registered library members along with their Member ID, name, membership type, and other relevant information. This feature helps the administrator verify and manage member records efficiently.

| Member ID | Name          | Email                    | Phone      | Borrowed / Limit | Outstanding Fine (\$) |
|-----------|---------------|--------------------------|------------|------------------|-----------------------|
| M004      | Diana Prince  | diana.p@university.edu   | 555-0104   | 0 / 4            | \$0.00                |
| M003      | Charlie Brown | charlie.b@university.edu | 555-0103   | 1 / 2            | \$0.00                |
| M002      | Bob Smith     | bob.smith@university.edu | 555-0102   | 1 / 3            | \$0.00                |
| M001      | Alice Johnson | alice.j@university.edu   | 555-0101   | 0 / 3            | \$5.00                |
| M006      | Selva Prabhu  | selvaprabhu21@gmail.com  | 9876543210 | 0 / 3            | \$0.00                |
| M005      | Dharshan      | dharshan2109@gmail.com   | 7305139670 | 0 / 3            | \$0.00                |

## Figure 11: Generate Reports

The Generate Reports module provides a summary of library activities, including issued books, returned books, available inventory, reservations, and overdue fines. These reports assist in monitoring library operations and evaluating resource utilization.

| <div>Circulation ReportInventory ReportFine ReportReserved Books Report</div> |                          |           |               |            |         |
|-------------------------------------------------------------------------------|--------------------------|-----------|---------------|------------|---------|
| CIRCULATION REPORT - Active Book Loans & Due Dates                            |                          |           |               |            |         |
| Book ID                                                                       | Title                    | Member ID | Member Name   | Due Date   | Status  |
| B104                                                                          | The Pragmatic Programmer | M003      | Charlie Brown | 2026-09-14 | On Time |
| B102                                                                          | Clean Code               | M002      | Bob Smith     | 2026-09-09 | On Time |
|                                                                               |                          |           |               |            |         |

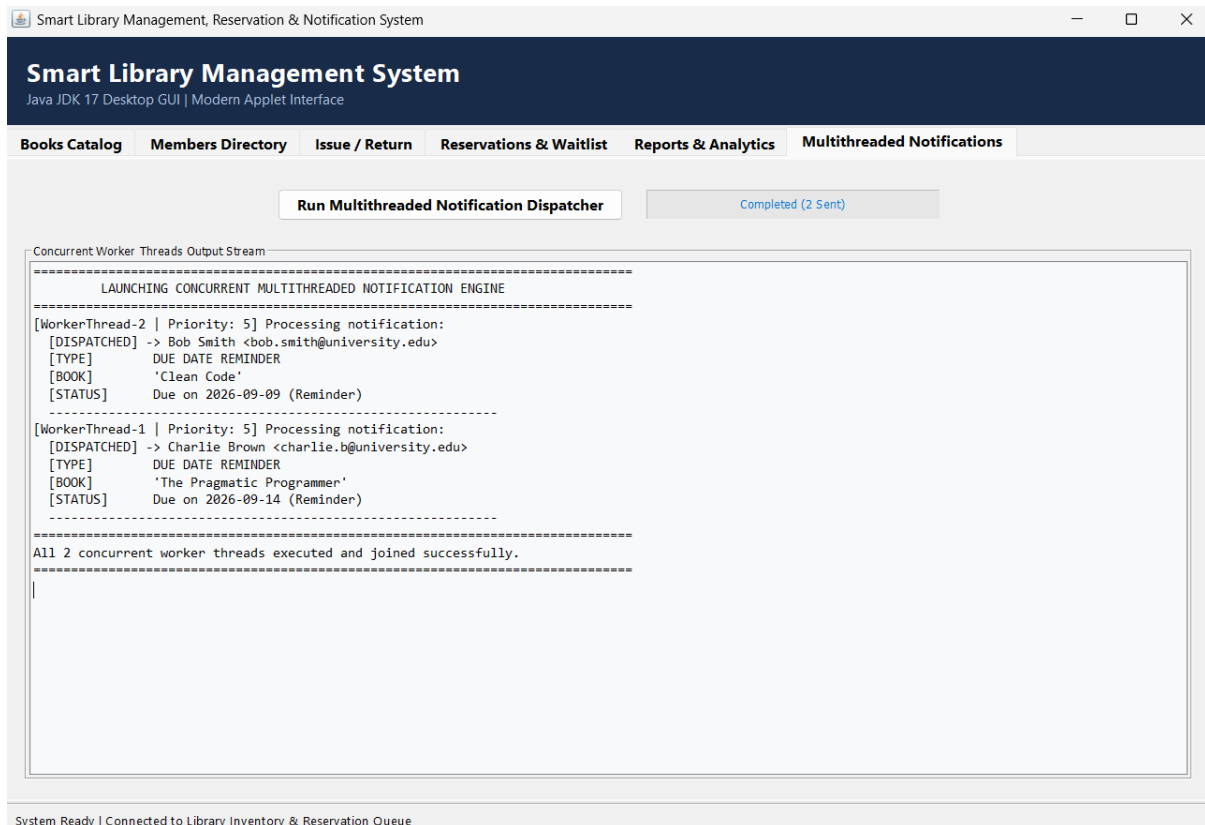
| INVENTORY REPORT - Catalog Statistics & Stock Levels |                              |                     |                   |              |           |               |
|------------------------------------------------------|------------------------------|---------------------|-------------------|--------------|-----------|---------------|
| Book ID                                              | Title                        | Author              | Category          | Total Copies | Available | Issued Copies |
| B109                                                 | Rich Dad Poor Son            | Dharshan            | Psychology        | 5            | 5         | 0             |
| B107                                                 | Modern Operating Syste...    | Andrew S. Tanenbaum | Operating Systems | 4            | 4         | 0             |
| B105                                                 | Artificial Intelligence      | Stuart Russell      | AI & Robotics     | 2            | 2         | 0             |
| B1068                                                | Human Antomy                 | Selva Prabhu        | Fun               | 250          | 250       | 0             |
| B106                                                 | Introduction to Algorithm... | Thomas H. Cormen    | Algorithms        | 1            | 1         | 0             |
| B103                                                 | Design Patterns              | Erich Gamma et al.  | Software Eng      | 2            | 2         | 0             |
| B104                                                 | The Pragmatic Programmm...   | Andy Hunt           | Software Eng      | 1            | 0         | 1             |
| B101                                                 | Effective Java               | Joshua Bloch        | Computer Science  | 3            | 3         | 0             |
| B102                                                 | Clean Code                   | Robert C. Martin    | Computer Science  | 2            | 1         | 1             |

| FINE REPORT - Outstanding Dues by Member |               |                        |          |                       |
|------------------------------------------|---------------|------------------------|----------|-----------------------|
| Member ID                                | Name          | Email                  | Phone    | Outstanding Fine (\$) |
| M001                                     | Alice Johnson | alice.j@university.edu | 555-0101 | \$5.00                |

| RESERVED BOOKS REPORT - Active Waitlist Queues |                          |                |                |                      |
|------------------------------------------------|--------------------------|----------------|----------------|----------------------|
| Book ID                                        | Book Title               | Queue Position | Reservation ID | Waiting Member       |
| B107                                           | Modern Operating Systems | #1             | RES-1001       | M005 (Dharshan)      |
| B104                                           | The Pragmatic Programmer | #1             | RES-1001       | M001 (Alice Johnson) |
| B104                                           | The Pragmatic Programmer | #2             | RES-1002       | M002 (Bob Smith)     |

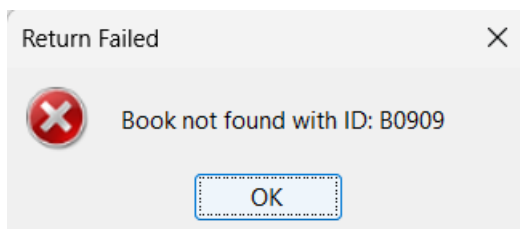
## Figure 12: Send Notifications

The Send Notifications module automatically generates due-date reminders and overdue alerts for members. Notifications are displayed through the console, helping members return books on time and improving overall library management.



## Figure 13: Exception Handling

The application validates user inputs before processing any operation. If an invalid Book ID, Member ID, duplicate reservation, or incorrect menu option is entered, the system displays an appropriate error message using Java exception handling mechanisms without terminating the application unexpectedly.



## 10. Results and Validation

| Module            | Result     |
|-------------------|------------|
| Book Management   | Successful |
| Member Management | Successful |
| Book Issue        | Successful |
| Book Return       | Successful |
| Reservation       | Successful |
| Reports           | Successful |
| Notifications     | Successful |

The application successfully performs all required operations. Testing confirmed that the implemented modules behave correctly, manage library records accurately, and generate the expected outputs, satisfying the assignment objectives.

## 11. Analysis, Comparison, Trade-offs and Justification

### Advantages

- Fast execution
- Easy navigation
- Modular code
- Efficient searching
- Object-oriented design
- Supports concurrent notifications

### Limitations

- GUI-based interface
- CSV files used as persistent database
- No database integration

## **Future Improvements**

- Java Swing or JavaFX GUI
- MySQL database (future enhancement from current CSV storage)
- Email notifications
- Barcode scanning
- Online reservation portal

## **12. Broader Considerations / SDG Relevance**

### **SDG 4 – Quality Education**

Provides efficient access to library resources and supports learning.

### **SDG 9 – Industry, Innovation and Infrastructure**

Uses modern software engineering concepts for digital library management.

### **SDG 16 – Peace, Justice and Strong Institutions**

Maintains accurate records and transparent circulation management.

## **13. Conclusion**

The Smart Library Management System successfully automates the daily operations of a library using Java. It demonstrates Object-Oriented Programming, Java Collections Framework, Exception Handling, and Multithreading while providing efficient book management, member registration, reservation handling, report generation, and notification services. The project satisfies the functional requirements of the assignment and provides a scalable foundation for future enhancements.



#### 14. Individual Contribution of Group Members

| Member           | Contribution                                                                                                                                                     |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mohan Raj V D    | Requirement analysis, project planning, GUI design, and documentation.                                                                                           |
| Monesh S         | Developed the Book Management module (Add, Update, Delete, Search Books) and implemented CSV file handling for book records.                                     |
| Selva Prabhu A K | Developed the Member Management module, Book Issue & Return module, and integrated CSV database operations for member and transaction records.                   |
| Ganesh           | Implemented the Reservation, Notification, and Report Generation modules. Assisted with exception handling and GUI event handling.                               |
| Dharshan A N     | Performed testing, debugging, validation, prepared screenshots, final report documentation, and coordinated project integration using Antigravity AI assistance. |

#### 15. References

1. Oracle Java SE Documentation
2. Java JDK 17 Documentation
3. Herbert Schildt, *Java: The Complete Reference*
4. Joshua Bloch, *Effective Java*
5. IntelliJ IDEA Documentation
6. Antigravity AI Documentation

## 16. One-Page Individual Reflection

**Ganesh P(192421142)**

Working on the **Smart Library Management, Book Reservation and Overdue Notification System** was a valuable experience that helped me connect the Java programming concepts learned in the course with a practical real-world application. My main contribution to the project was related to **book management and CSV database integration**. I worked on the functionality required to store, retrieve, search, and manage book information while ensuring that the records were maintained correctly in the CSV files.

Through this project, I gained a better understanding of **Object-Oriented Programming**, particularly the use of classes, objects, methods, encapsulation, and modular program design. Implementing the book management functionality helped me understand how individual components can work together as part of a larger software system. I also gained practical experience in **file handling**, especially reading data from CSV files, writing updated records, and maintaining persistent data. This was useful because the project uses CSV files as its database rather than an external database management system.

Another important learning experience was working with the **Java Applet GUI**. I learned how graphical components such as buttons, text fields, labels, tables, and dialog boxes can be connected with Java event-handling mechanisms. This made me understand the difference between a traditional console-based program and an event-driven GUI application. I also learned the importance of validating user input before performing operations so that incorrect or incomplete information does not affect the stored records.

During development, I faced challenges while handling CSV records and ensuring that changes made through the GUI were correctly reflected in the stored data. Debugging these issues improved my logical thinking and problem-solving ability. I learned to test individual functions carefully before integrating them with other modules. I also understood that even a small change in one module can affect other parts of the application, making proper testing and coordination important.

**Antigravity AI** was used as a development assistant during the project. It helped us generate code structures, identify possible implementation approaches, and resolve some programming difficulties. However, I understood that AI-generated code should not simply be copied without

verification. I reviewed, modified, and tested the code to understand how each part worked and to ensure that it matched our project requirements.

Working with my teammates also improved my **communication, collaboration, and coordination skills**. We divided the project into different modules and discussed how each component should interact with the others. This taught me how teamwork is important when developing a larger application and how proper integration is necessary to produce a working final system.

Overall, this assignment improved my understanding of **Java programming, GUI development, CSV file handling, debugging, testing, and teamwork**. It gave me practical experience in developing a complete software application and increased my confidence in applying programming concepts to real-world problems.